Scott Morel

MLAS 500

Final Project

For my final project, I decided to create a sentiment analysis program using PyTorch and VADER library. The first step I took was to download and initialize all the required modules. After that I downloaded the VADER lexicon (https://github.com/cjhutto/vaderSentiment) and set the program to use GPU if available for faster processing.

```
#Install modules
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import pandas as pd
import numpy as np
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from sklearn.model_selection import train_test_split
from collections import Counter
import re

#Download VADER
nltk.download('vader_lexicon')

# Set device (GPU if available, if not CPU)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

My next step was to create a dataset of various statements for scoring. From there I initialized the VADER library and defined the function to get labels for the statements. Finally, I applied VADER scoring to the statements. Positive statements got assigned a label of 1, while negative statements got a label of 0.

```python
#Create dataset
data = {
    'text': [
        "I absolutely love this movie, it's fantastic!",
        "This was the worst food I have ever eaten.",
        "Horrid!  Never again.",
        "Phenominal product!  I highly recommend.",
        "The plot was okay, but the acting was terrible.",
        "I am so happy with this product.",
        "Really disappointed with the service.",
        "An average experience, nothing special.",
    ]
}

df = pd.DataFrame(data)

#Initialize VADER
analyzer = SentimentIntensityAnalyzer()

#Define function to get labels
def get_vader_label(text):
    scores = analyzer.polarity_scores(text)
# Score > 0.05 is positive, otherwise negative
    return 1 if scores['compound'] >= 0.05 else 0

#Apply VADER to dataset
df['vader_score'] = df['text'].apply(lambda x: analyzer.polarity_scores(x)['compound'])
df['label'] = df['text'].apply(get_vader_label)

print("Data labeled by VADER:")
print(df)
```

```
Data labeled by VADER:
                                          text  vader_score  label
0    I absolutely love this movie, it's fantastic!       0.8550      1
1       This was the worst food I have ever eaten.      -0.6249      0
2                           Horrid!  Never again.      -0.5848      0
3         Phenominal product!  I highly recommend.       0.4740      1
4  The plot was okay, but the acting was terrible.      -0.5719      0
5                I am so happy with this product.       0.6115      1
6            Really disappointed with the service.      -0.5256      0
7          An average experience, nothing special.      -0.3089      0
```

Next, I created the function for tokenization of each statement, as well as creating the vocabulary mapping. I then created a function to convert the text to number strings and created the features and labels. From there I converted to Tensor and created the data loader for the train and test sets.

```
#Tokenization
def tokenize(text):
    return re.sub(r'[^a-z ]', '', text.lower()).split()

all_words = []
for text in df['text']:
    all_words.extend(tokenize(text))

#Create vocabulary mapping
vocab_count = Counter(all_words)
vocab = sorted(vocab_count, key=vocab_count.get, reverse=True)
word_to_idx = {word: i+1 for i, word in enumerate(vocab)}
word_to_idx['<PAD>'] = 0

print(f"Vocabulary Size: {len(word_to_idx)}")
print(f"Sample mapping: {list(word_to_idx.items())[:5]}")

#Convert text to string of numbers
def text_to_indices(text, word_to_idx, max_len=10):
    tokens = tokenize(text)
    indices = [word_to_idx.get(token, 0) for token in tokens]

    if len(indices) < max_len:
        indices += [0] * (max_len - len(indices))
    else:
        indices = indices[:max_len]
    return indices

#Create features and labels
X = [text_to_indices(t, word_to_idx) for t in df['text']]
y = df['label'].values

#Convert to tensor
X_tensor = torch.tensor(X, dtype=torch.long)
y_tensor = torch.tensor(y, dtype=torch.float32)

#Create dataloader for train and test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_tensor, y_tensor, test_size=0.2, random_state=42)

print(f"Shape of X: {X_tensor.shape}")
print(f"Shape of y: {y_tensor.shape}")
```

After that, I created the sentiment model function and associated parameters. After that, I created the training loop and evaluation mode. I set up the status to give any statements with a VADER score greater than or equal to 0.5 a positive score, a neutral score for anything between a VADER score of 0 and 0.5, and a negative score to any VADER score

less than 0. Then I typed up various statements to test

```
#Sentiment model function
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super(SentimentLSTM, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)

        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)

        self.fc = nn.Linear(hidden_dim, output_dim)

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        embedded = self.embedding(x)

        _, (hidden, _) = self.lstm(embedded)

        hidden = hidden.squeeze(0)

        out = self.fc(hidden)
        return self.sigmoid(out)

#Parameters
VOCAB_SIZE = len(word_to_idx) + 1
EMBEDDING_DIM = 32
HIDDEN_DIM = 64
OUTPUT_DIM = 1

model = SentimentLSTM(VOCAB_SIZE, EMBEDDING_DIM, HIDDEN_DIM, OUTPUT_DIM).to(device)
print(model)
```

```
SentimentLSTM(
  (embedding): Embedding(40, 32, padding_idx=0)
  (lstm): LSTM(32, 64, batch_first=True)
  (fc): Linear(in_features=64, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)
```

```
)
```

5]:

```
#Loss and optimizer setup
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

#Training loop
epochs = 5
model.train()

for epoch in range(epochs):

    inputs, labels = X_tensor.to(device), y_tensor.to(device)

    optimizer.zero_grad()

    outputs = model(inputs).squeeze()

    loss = criterion(outputs, labels)

    loss.backward()

    optimizer.step()

    if (epoch+1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')

print("Training Complete!")
```

```
Training Complete!
```

```
#Set evaluation mode
def predict_sentiment(text):
    model.eval()

    processed_input = text_to_indices(text, word_to_idx)
    input_tensor = torch.tensor([processed_input], dtype=torch.long).to(device)

#Get VADER score
    vader_score = analyzer.polarity_scores(text)['compound']
#Set status
    status = "POSITIVE" if vader_score >= 0.5 else ("NEUTRAL" if vader_score > 0 else "NEGATIVE")

    print(f"Text: \"{text}\"")
    print(f"VADER Score: {vader_score:.4f}) ({status})")
    print("-" * 30)

#Test model
predict_sentiment("I loved the movie, it was great!")
predict_sentiment("The food was absolutely terrible and cold.")
predict_sentiment("I am not sure how I feel about this.")
predict_sentiment("Why even bother?")
predict_sentiment("Phenominal! Perfect execution!")
predict_sentiment("Could have been better but not bad.")
predict_sentiment("Terrible service.  Look elsewhere.")

#Testing for effect of punctuation
predict_sentiment("Absolute garbage!  Waste of time.")
predict_sentiment("Absolute garbage.  Waste of time.")
predict_sentiment("Meh.")
predict_sentiment("Meh!")
predict_sentiment("Decent enough.  Can't complain.")
predict_sentiment("Decent enough.  Can't complain!")
predict_sentiment("Decent enough!  Can't complain!")
```

```
Text: "I loved the movie, it was great!"
VADER Score: 0.8516) (POSITIVE)
------------------------------
Text: "The food was absolutely terrible and cold."
VADER Score: -0.5256) (NEGATIVE)
------------------------------
Text: "I am not sure how I feel about this."
VADER Score: -0.2411) (NEGATIVE)
------------------------------
Text: "Why even bother?"
VADER Score: -0.3400) (NEGATIVE)
------------------------------
Text: "Phenominal! Perfect execution!"
VADER Score: 0.6467) (POSITIVE)
------------------------------
Text: "Could have been better but not bad."
VADER Score: 0.6932) (POSITIVE)
------------------------------
Text: "Terrible service.  Look elsewhere."
VADER Score: -0.4767) (NEGATIVE)
------------------------------
Text: "Absolute garbage!  Waste of time."
VADER Score: -0.4753) (NEGATIVE)
------------------------------
Text: "Absolute garbage.  Waste of time."
VADER Score: -0.4215) (NEGATIVE)
------------------------------
Text: "Meh."
VADER Score: -0.0772) (NEGATIVE)
------------------------------
Text: "Meh!"
VADER Score: -0.1511) (NEGATIVE)
------------------------------
Text: "Decent enough.  Can't complain."
VADER Score: 0.2755) (NEUTRAL)
------------------------------
Text: "Decent enough.  Can't complain!"
VADER Score: 0.3404) (NEUTRAL)
------------------------------
Text: "Decent enough!  Can't complain!"
VADER Score: 0.4007) (NEUTRAL)
------------------------------
```

Overall, I'd say it is a pretty accurate program for sentiment analysis. It did a great job of determining positive, negative, and neutral statements. I also noticed that punctuation plays a part in the scoring.